

A&I

android and ios develop.

MNIST

2026 A&I 4th

AI 멘토 구승민

목차

1. 코드 실습

- MNIST 데이터셋을 통해 학습하는 모델 만들기

2. 코드 설명

- 실습에서 활용된 코드의 동작 이해하기

3. EDA, 전처리, Feature Engineering

- 데이터 처리에 따른 모델의 결과 확인하기

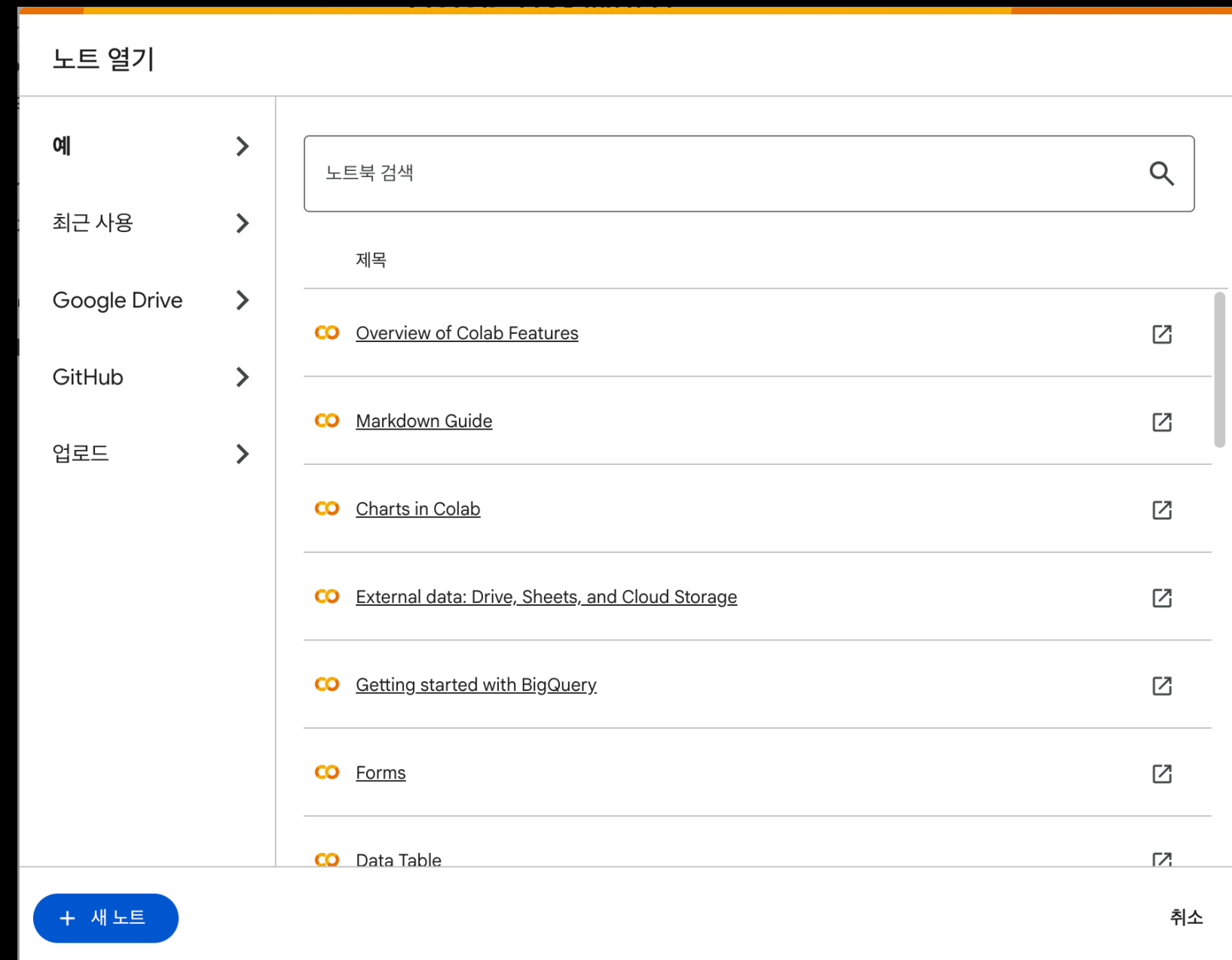
1. 코드 실습

- 그냥 따라하시면 됩니다

실습 환경 구성하기

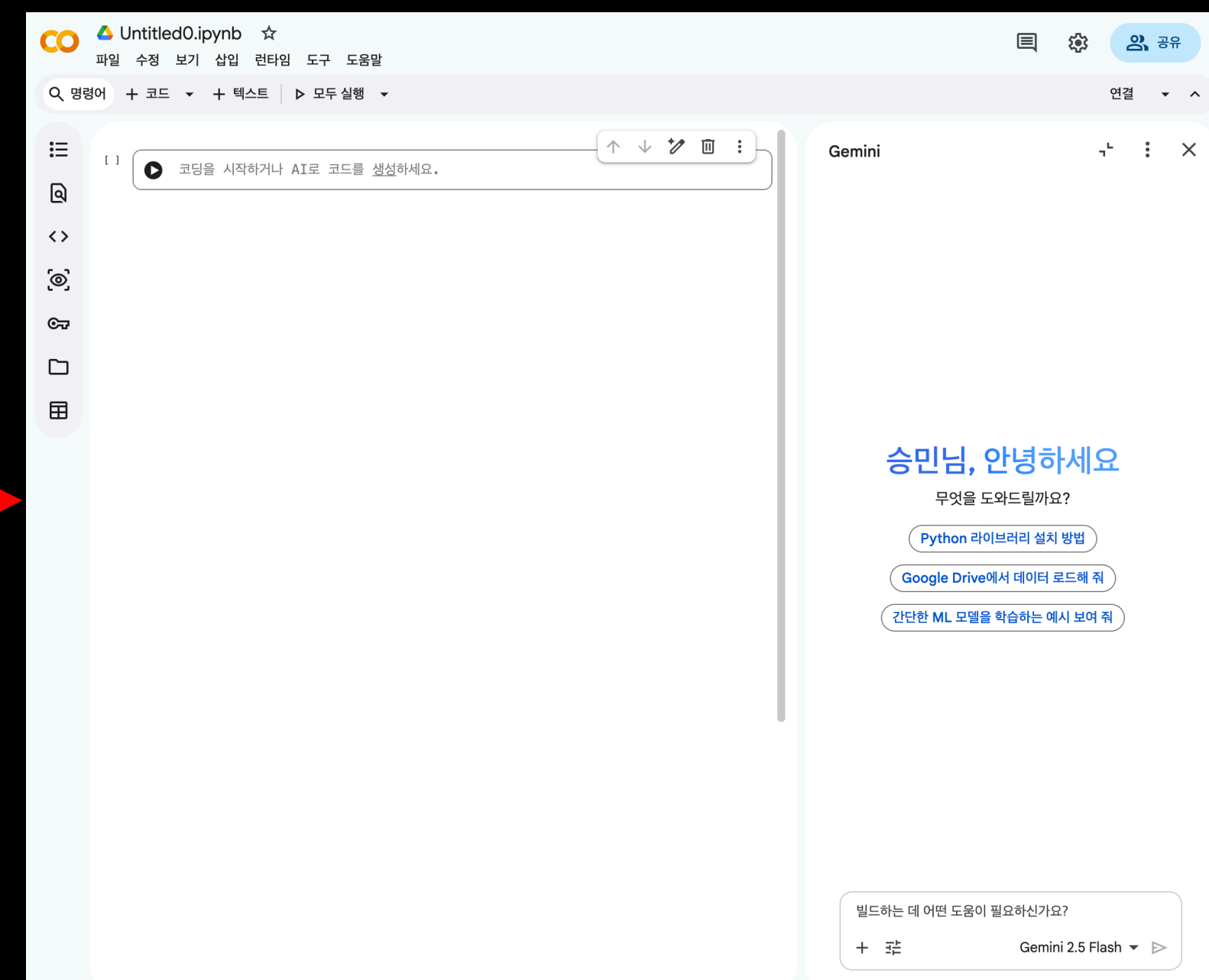
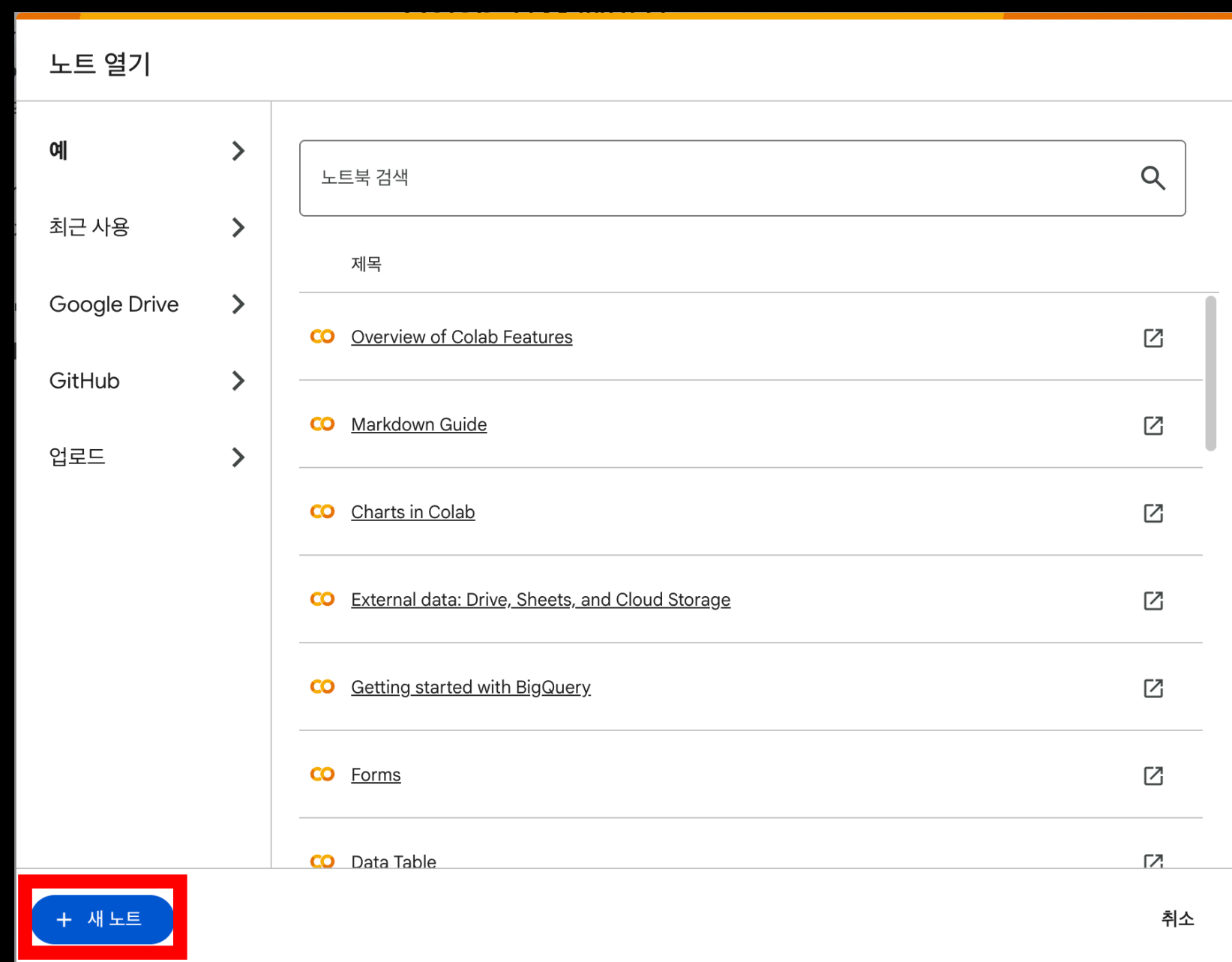
아래 링크로 이동

- <https://colab.research.google.com>



실습 환경 구성하기

하단 새 노트 클릭하여 코드 환경 구성



코랩 단축키

[실행 관련 단축키]

1. Command + Enter

- 결과 값만 보고자 할 때
- 해당 셀을 실행하고 커서를 해당 셀에 두는 경우

2. Shift + Enter

- 여러가지 값을 빠르게 출력할 때
- 해당 셀을 실행하고 커서를 다음 셀로 넘기는 경우

3. Alt + Enter

- 다음 작업 공간이 없을 때
- 해당 셀을 실행하고 셀을 삽입한 후 커서를 삽입한 셀로 넘기는 경우

코랩 단축키

[셀 삽입/삭제 관련 단축키]

1. Command + M D

- 셀 지우기

2. Command + M Y

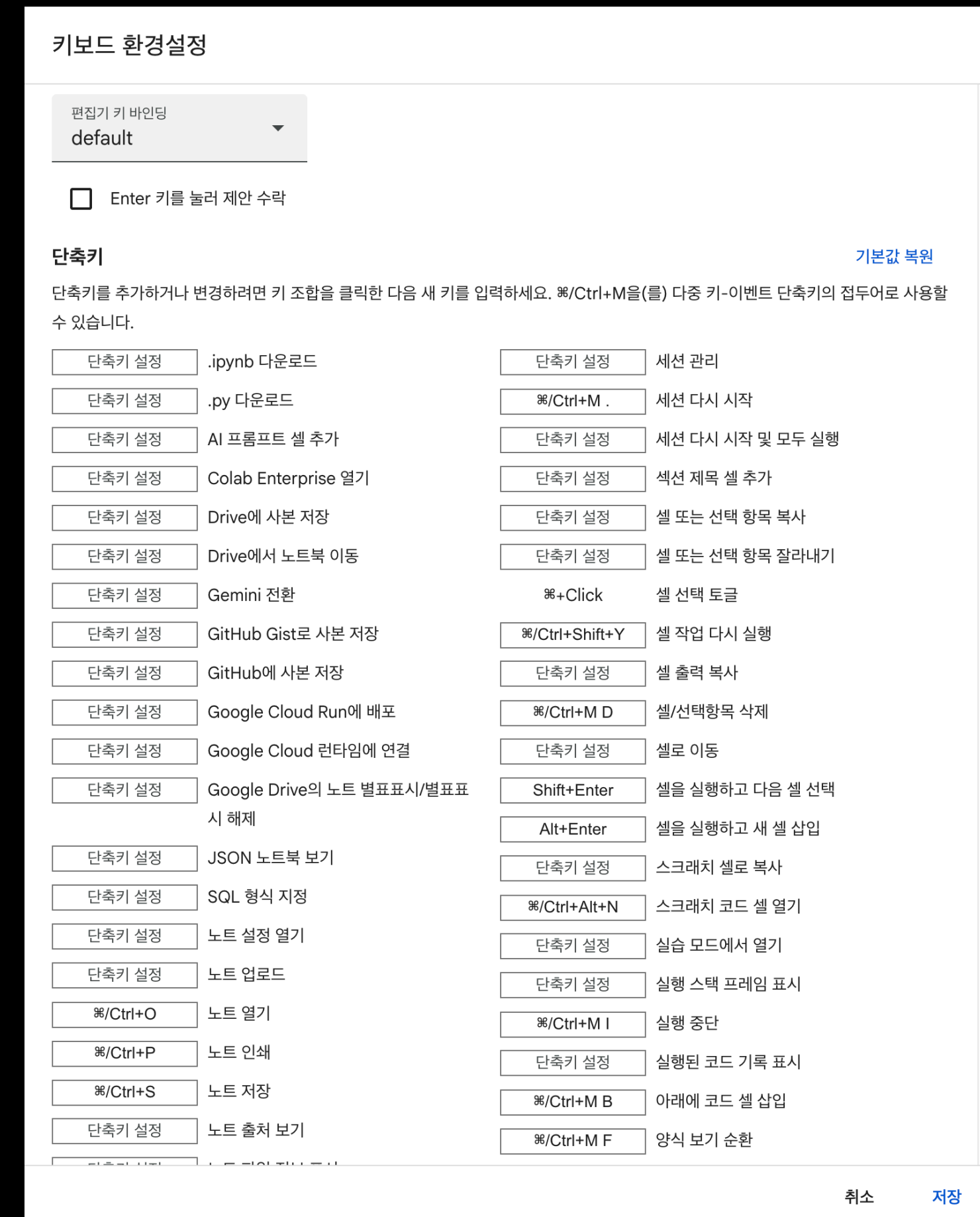
- 코드 셀로 변경

3. Command + M M

- 마크다운 셀로 변경

4. Command + M H

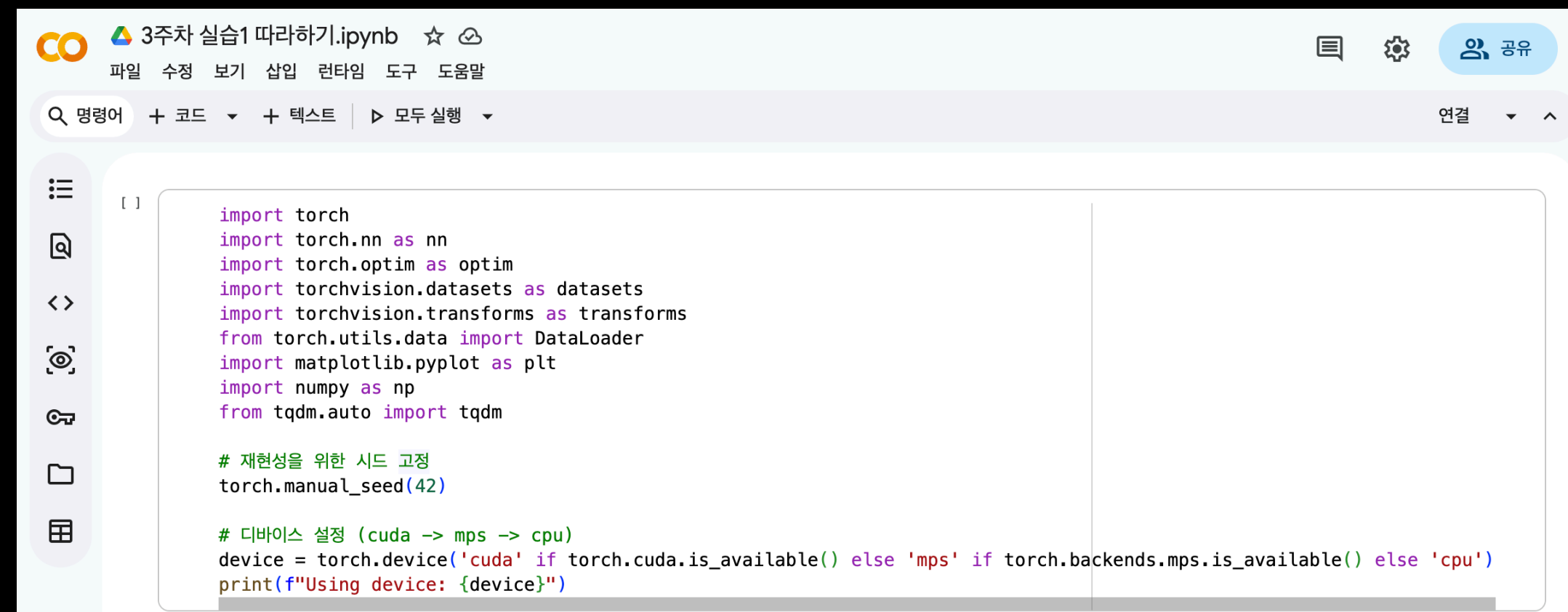
- 단축키 모음 열기



코드 실습

코드 내용을 따라하며 실습

- 중간에 놓치는 경우 아래 링크에서 내용을 확인
- <https://colab.research.google.com/drive/1vI6dHFcBJw9rxRStc2pG3sdhftO2ZNs3?usp=sharing>



```
[ ]  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
import torchvision.datasets as datasets  
import torchvision.transforms as transforms  
from torch.utils.data import DataLoader  
import matplotlib.pyplot as plt  
import numpy as np  
from tqdm.auto import tqdm  
  
# 재현성을 위한 시드 고정  
torch.manual_seed(42)  
  
# 디바이스 설정 (cuda -> mps -> cpu)  
device = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else 'cpu')  
print(f"Using device: {device}")
```

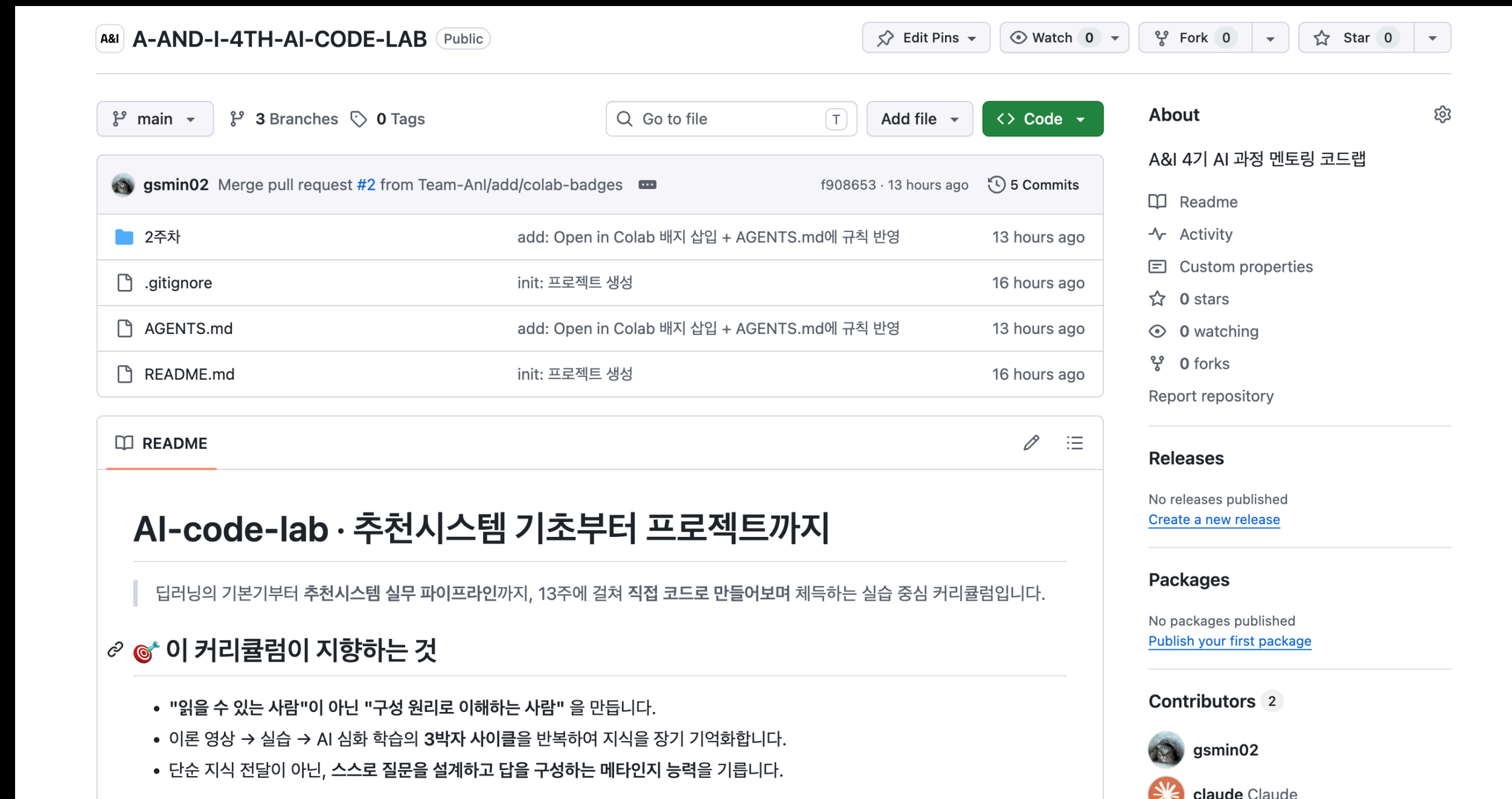
2. 코드 설명

- 작성한 코드의 세부 내용을 살펴봅니다

강의자료 확인 방법

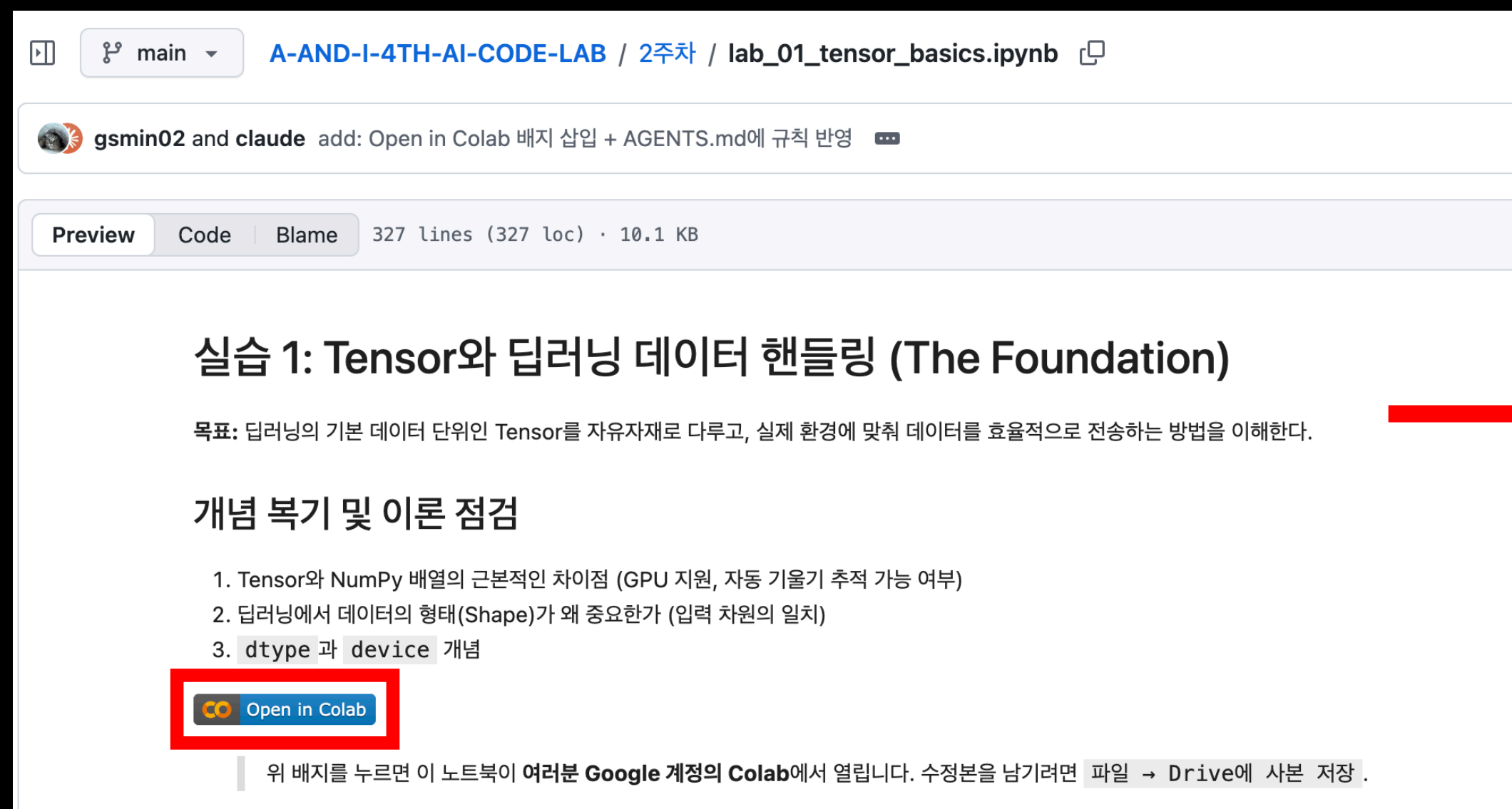
강의자료는 아래 링크를 통해 확인 가능

- <https://github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB>

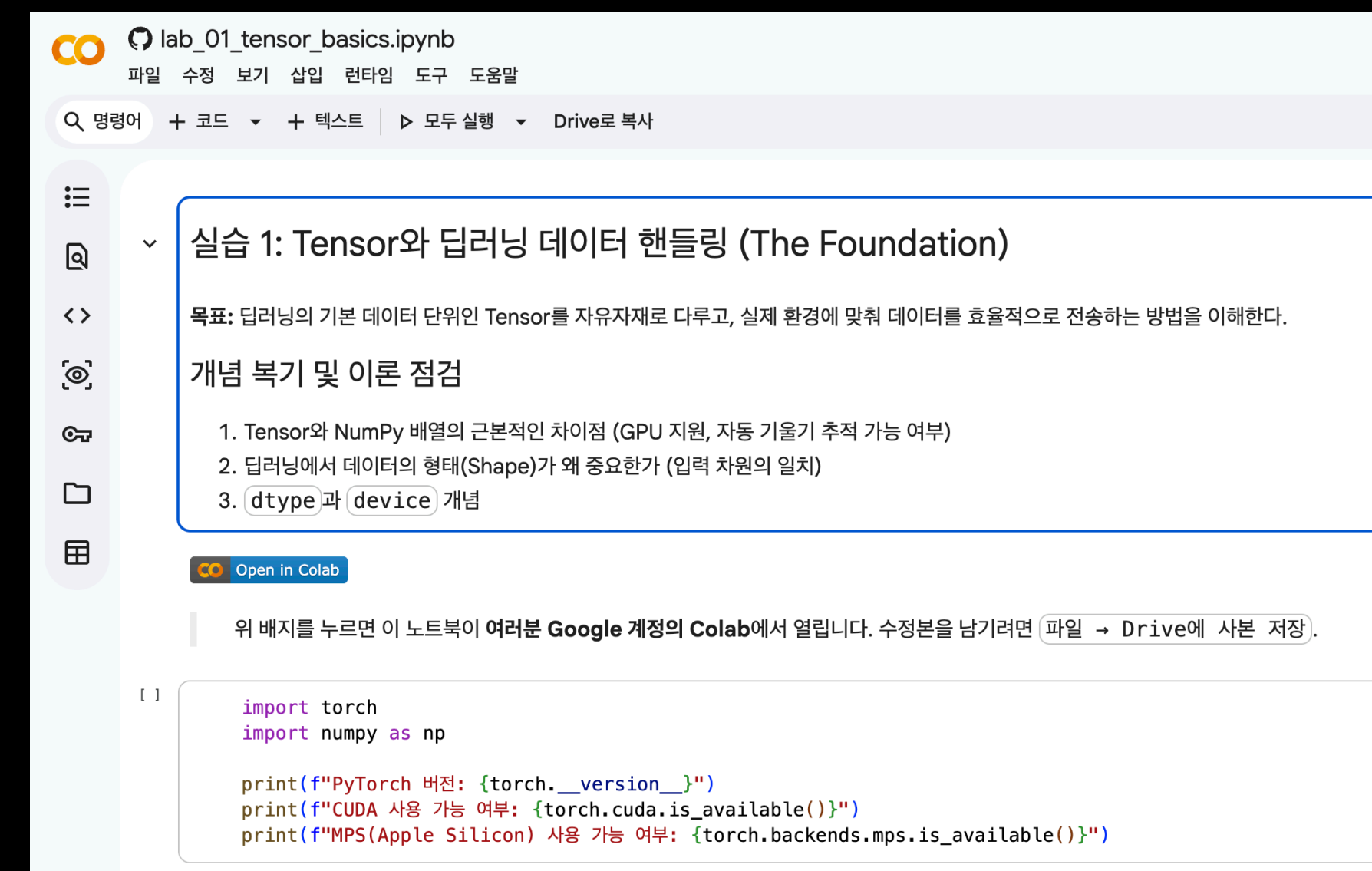


코드 실습 수행

각 실습은 제공된 ipynb 파일 상단 “**Open in Colab**”으로 실습 가능



The screenshot shows a GitHub repository page for the file 'lab_01_tensor_basics.ipynb'. The file is 327 lines (327 loc) and 10.1 KB. The 'Preview' tab is selected, showing the title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)'. Below the title is a description: '목표: 딥러닝의 기본 데이터 단위인 Tensor를 자유자재로 다루고, 실제 환경에 맞춰 데이터를 효율적으로 전송하는 방법을 이해한다.' followed by a section '개념 복기 및 이론 점검' with three numbered points. At the bottom, there is a red box highlighting the 'Open in Colab' button.



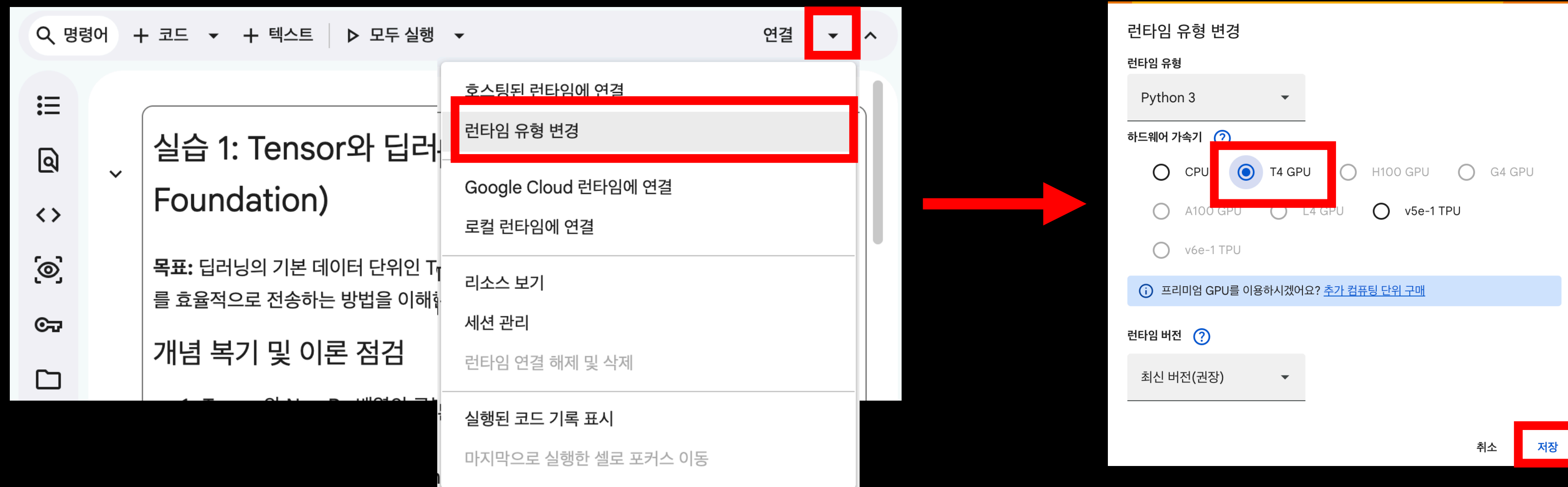
The screenshot shows the Google Colab interface for the file 'lab_01_tensor_basics.ipynb'. The title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)' is at the top. Below it is the same description and '개념 복기 및 이론 점검' section as in the GitHub preview. A red arrow points from the 'Open in Colab' button in the GitHub preview to the 'Open in Colab' button in the Colab interface. The Colab interface also shows the 'Open in Colab' button and a section for '위 배치를 누르면 이 노트북이 여러분 Google 계정의 Colab에서 열립니다.' followed by a code cell with the following code:

```
import torch
import numpy as np

print(f"PyTorch 버전: {torch.__version__}")
print(f"CUDA 사용 가능 여부: {torch.cuda.is_available()}")
print(f"MPS(Apple Silicon) 사용 가능 여부: {torch.backends.mps.is_available()}")
```

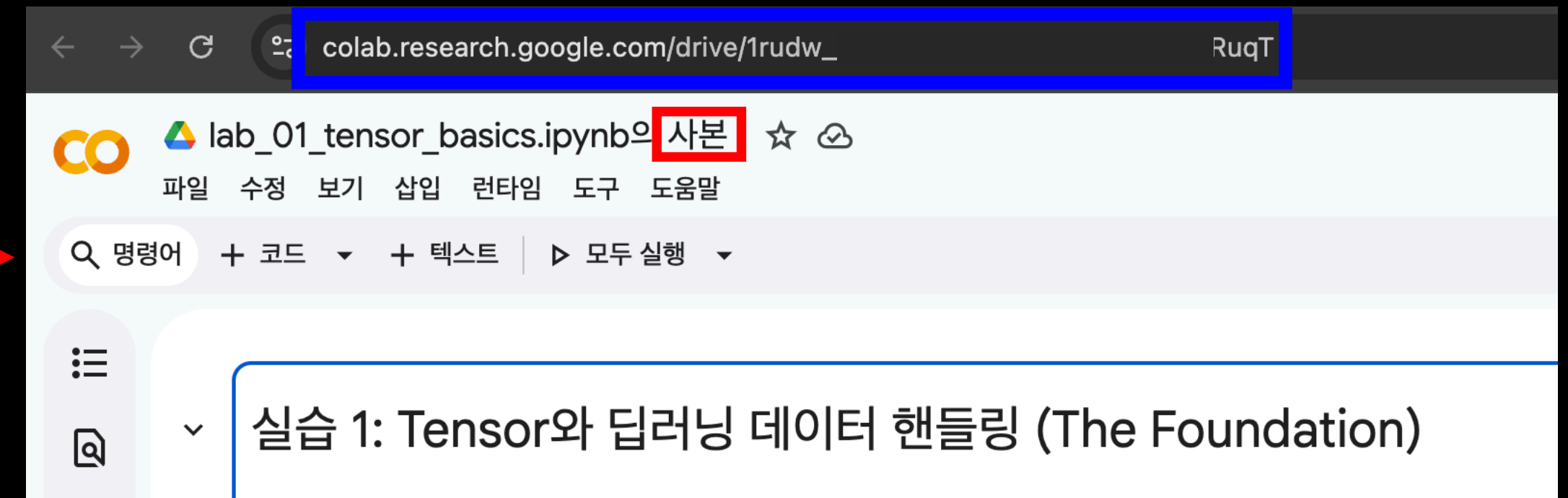
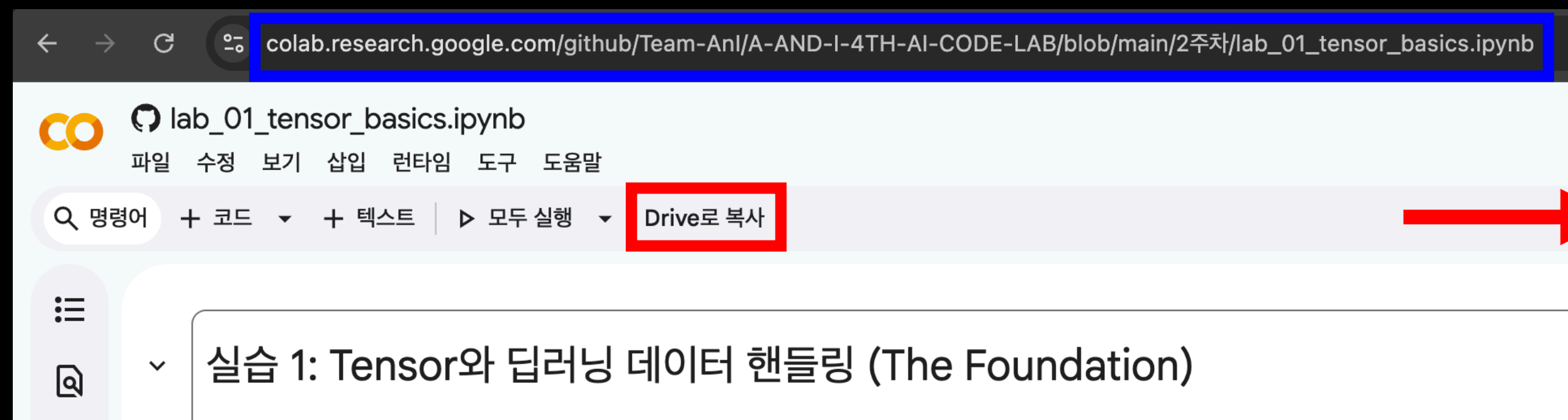
코드 실습 주의사항

실행 전 “런타임 유형 변경”을 통해 “T4 GPU”로 변경해야 함



내용 편집

내용을 변경하고 싶은 경우 “**Drive로 복사**”를 통해 편집 가능



1-1. 환경 준비: 라이브러리 импорт

먼저 실습에 필요한 라이브러리들을 불러옵니다. lab_01 과 동일한 구성입니다.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
from tqdm.auto import tqdm
```

Python

코드 해설

- `torch`: 핵심 PyTorch 라이브러리. 텐서 및 기본 연산을 다룹니다.
- `torch.nn`: 신경망을 구성하기 위한 모듈, 레이어, 활성화 함수, 손실 함수 등이 들어있습니다. (`nn.Module`, `nn.Linear` 등)
- `torch.optim`: 경사 하강법을 수행하는 최적화 알고리즘(Optimizer)들이 들어있습니다. (`optim.Adam`, `optim.SGD` 등)
- `torchvision`: 이미지 처리를 위한 PyTorch의 공식 라이브러리. 유명 데이터셋, 모델 아키텍처, 이미지 변환 도구를 포함합니다.
- `DataLoader`: 데이터셋을 미니배치 형태로 효율적으로 모델에 공급하는 역할을 합니다.
- `matplotlib.pyplot`: 데이터와 학습 결과를 시각화하기 위한 표준 라이브러리입니다.
- `tqdm.auto`: 반복문(loop)의 진행 상황을 시각적인 막대로 보여주어 학습 과정을 모니터링하기 용이하게 합니다.

1-2. 환경 준비: 재현성 및 장치 설정

실험 결과를 재현 가능하게 만들고, GPU 같은 하드웨어 가속을 사용하도록 설정합니다.

```
# 재현성을 위한 시드 고정
torch.manual_seed(42)

# 디바이스 설정 (cuda -> mps -> cpu)
device = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else 'cpu')
print(f"Using device: {device}")
```

Python

코드 해설

- `torch.manual_seed(42)`: 난수 생성기의 시드를 고정합니다. 딥러닝에서는 모델의 초기 가중치 설정, 데이터 셔플링 등 무작위성이 개입하는 부분이 많습니다. 시드를 고정하면 코드를 다시 실행해도 항상 동일한 결과를 얻을 수 있어 실험의 **재현성**을 보장합니다.
- `device`: 코드를 실행할 장치를 결정합니다. NVIDIA GPU(`cuda`)가 있으면 최우선으로 사용하고, 없으면 Apple Silicon GPU(`mps`)를, 둘 다 없으면 `cpu`를 사용하도록 설정합니다. 텐서나 모델에 `.to(device)`를 붙여주면 해당 장치로 데이터가 옮겨져 연산이 가속됩니다.

2-1. 데이터 전처리: Transform 정의

이미지 데이터를 모델이 학습할 수 있는 텐서 형태로 변환하는 규칙을 정의합니다. 이 부분이 데이터 파이프라인의 핵심 중 하나입니다.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

Python

코드 해설

- `transforms.Compose([...])`: 여러 단계의 변환(transform)을 순서대로 실행하는 파이프라인을 만듭니다. 위에서 아래로 순차적으로 적용됩니다.
- 1. `transforms.ToTensor()`: 이 한 줄은 사실 세 가지 중요한 일을 동시에 처리합니다.
 1. 입력 데이터를 PyTorch 텐서로 변환합니다. (e.g., PIL Image나 NumPy 배열 -> `torch.Tensor`)
 2. 이미지 픽셀 값을 `[0, 255]` 범위에서 `[0.0, 1.0]` 범위로 스케일링합니다. ($x / 255.0$)
 3. 이미지 텐서의 차원 순서를 `(H, W, C)` (높이, 너비, 채널)에서 PyTorch 표준인 `(C, H, W)` 로 재배열합니다.
- 2. `transforms.Normalize((0.1307,), (0.3081,))`: 텐서의 값을 정규화(표준화)합니다. `output = (input - mean) / std` 공식을 따릅니다.
 - `(0.1307,)` 과 `(0.3081,)` 은 각각 MNIST 학습 데이터셋 전체의 픽셀 값 평균(mean)과 표준편차(std)를 미리 계산해 둔 값입니다. 이렇게 정규화를 수행하면 데이터의 분포가 평균 0, 표준편차 1에 가깝게 조정되어 모델이 더 안정적이고 빠르게 학습하는 데 도움이 됩니다. (마치 모든 특성의 단위를 맞춰주는 것과 같습니다.)
 - MNIST는 흑백 이미지라 채널이 1개이므로, 평균과 표준편차를 하나의 값만 가진 튜플 `(value,)` 형태로 전달합니다.

2-2. 데이터 로드: Dataset과 DataLoader

정의된 `transform`을 적용하여 MNIST 데이터셋을 불러오고, 이를 배치(batch) 단위로 묶어주는 `DataLoader`를 생성합니다.

```
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=1000, shuffle=False)
```

Python

코드 해설

- `datasets.MNIST(...)`: `torchvision`에서 제공하는 MNIST 데이터셋 클래스입니다.
 - `root='./data'`: 데이터가 저장될 로컬 경로입니다.
 - `train=True/False`: `True`이면 60,000개의 학습용 데이터를, `False`이면 10,000개의 평가용 데이터를 불러옵니다. 모델의 성능을 공정하게 평가하기 위해 학습에 쓰이지 않은 데이터로 검증하는 것은 매우 중요합니다.
 - `download=True`: 지정된 경로에 데이터가 없으면 자동으로 다운로드합니다.
 - `transform=transform`: 앞에서 정의한 전처리 파이프라인을 지정합니다. 데이터셋에서 이미지를 하나씩 꺼낼 때마다 이 `transform`이 적용됩니다.
- `DataLoader(...)`: `Dataset`을 감싸서 미니배치(mini-batch)를 만들어주는 역할을 합니다.
 - `dataset=...`: 사용할 데이터셋을 지정합니다.
 - `batch_size=64`: 한 번에 모델에 공급할 데이터의 개수입니다. 전체 데이터를 한 번에 처리하는 것보다 메모리 효율적이며, 학습 안정성에도 기여합니다.
 - `shuffle=True`: 학습용 로더에만 `True`로 설정합니다. 매 에폭(epoch)마다 데이터의 순서를 섞어주어 모델이 데이터의 순서 자체를 외우는 것을 방지하고 일반화 성능을 높입니다. 평가용 로더는 굳이 섞을 필요가 없습니다.

Step 1

- Defines how data is loaded
- Has `__getitem__` method

Custom
Dataset class

Instantiate

Training dataset

Validation dataset

Test dataset

Step 2

- Parent class imported from PyTorch

DataLoader class

Instantiate

Train dataloader

Validation dataloader

Test dataloader

Step 3

- Define
 - shuffling
 - batch size
 - number of processes
 - and much more

3. 모델 정의: 다층 퍼셉트론 (MLP)

이제 데이터를 처리할 신경망 모델을 설계합니다. PyTorch에서는 `nn.Module` 을 상속받아 모델 클래스를 정의합니다.

```
class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

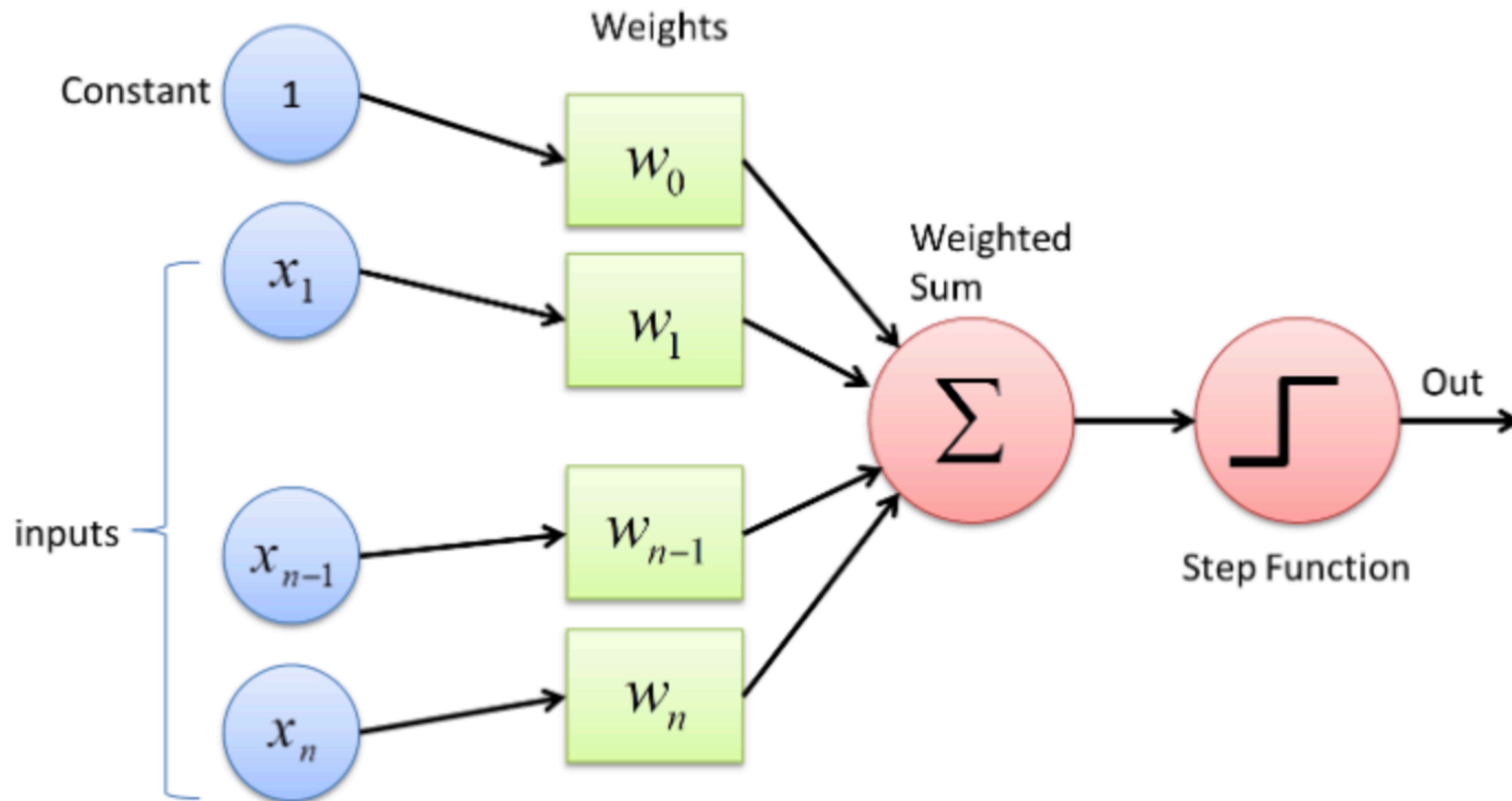
    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

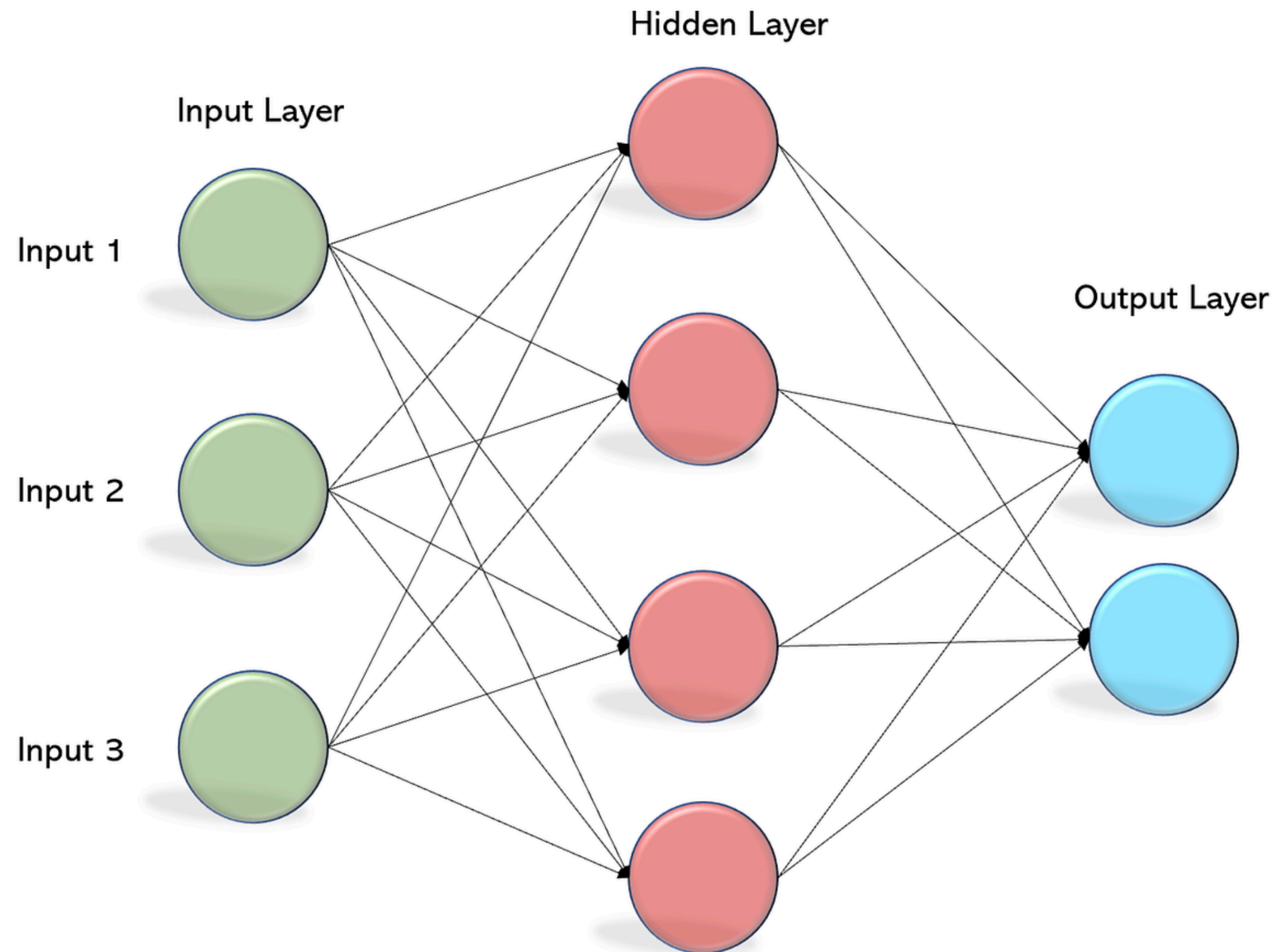
model = SimpleMLP().to(device)
print(model)
```

Python

코드 해설

- **`class SimpleMLP(nn.Module)`**: 모든 PyTorch 모델은 `nn.Module` 을 부모 클래스로 상속받습니다. 이를 통해 모델의 파라미터(가중치, 편향)를 자동으로 추적하고 관리하는 등 다양한 편의 기능을 사용할 수 있습니다.
- **`__init__(self)`**: 모델이 사용할 레이어(layer)들을 정의하는 생성자입니다.
 - `super().__init__()`: 부모 클래스(`nn.Module`)의 생성자를 반드시 먼저 호출해야 합니다.
 - `nn.Flatten()`: (배치 크기, 1, 28, 28) 모양의 4차원 텐서를 (배치 크기, 784) 모양의 2차원 텐서로 짝 펴줍니다. MLP의 `nn.Linear` 레이어는 1차원 벡터 입력을 받기 때문에 이 과정이 필수적입니다.
 - `nn.Linear(28 * 28, 128)`: 첫 번째 완전 연결(Fully Connected) 레이어입니다. 입력 특징(feature) 784개(28x28)를 받아 128개의 출력 특징으로 변환합니다.
 - `nn.ReLU()`: 활성화 함수(Activation Function)입니다. 입력값이 0보다 작으면 0으로, 0보다 크면 그대로 출력합니다. 이 비선형(non-linear) 변환이 없다면 여러 개의 레이어를 쌓는 의미가 없어집니다. (선형 변환을 여러 번 해도 결국 하나의 선형 변환과 같기 때문)
 - `nn.Linear(128, 10)`: 두 번째 완전 연결 레이어이자 출력 레이어입니다. 이전 레이어에서 받은 128개 특징을 최종적으로 10개(0~9 숫자 클래스)의 점수(logit)로 변환합니다.
- **`forward(self, x)`**: 데이터가 모델을 통과하는 순서(흐름)를 정의합니다. `__init__` 에서 정의한 레이어들을 여기에 순서대로 적용합니다. 모델 객체를 함수처럼 호출하면 (`model(data)`) 이 `forward` 메소드가 자동으로 실행됩니다.





4-1. 훈련 준비: 손실 함수와 옵티마이저

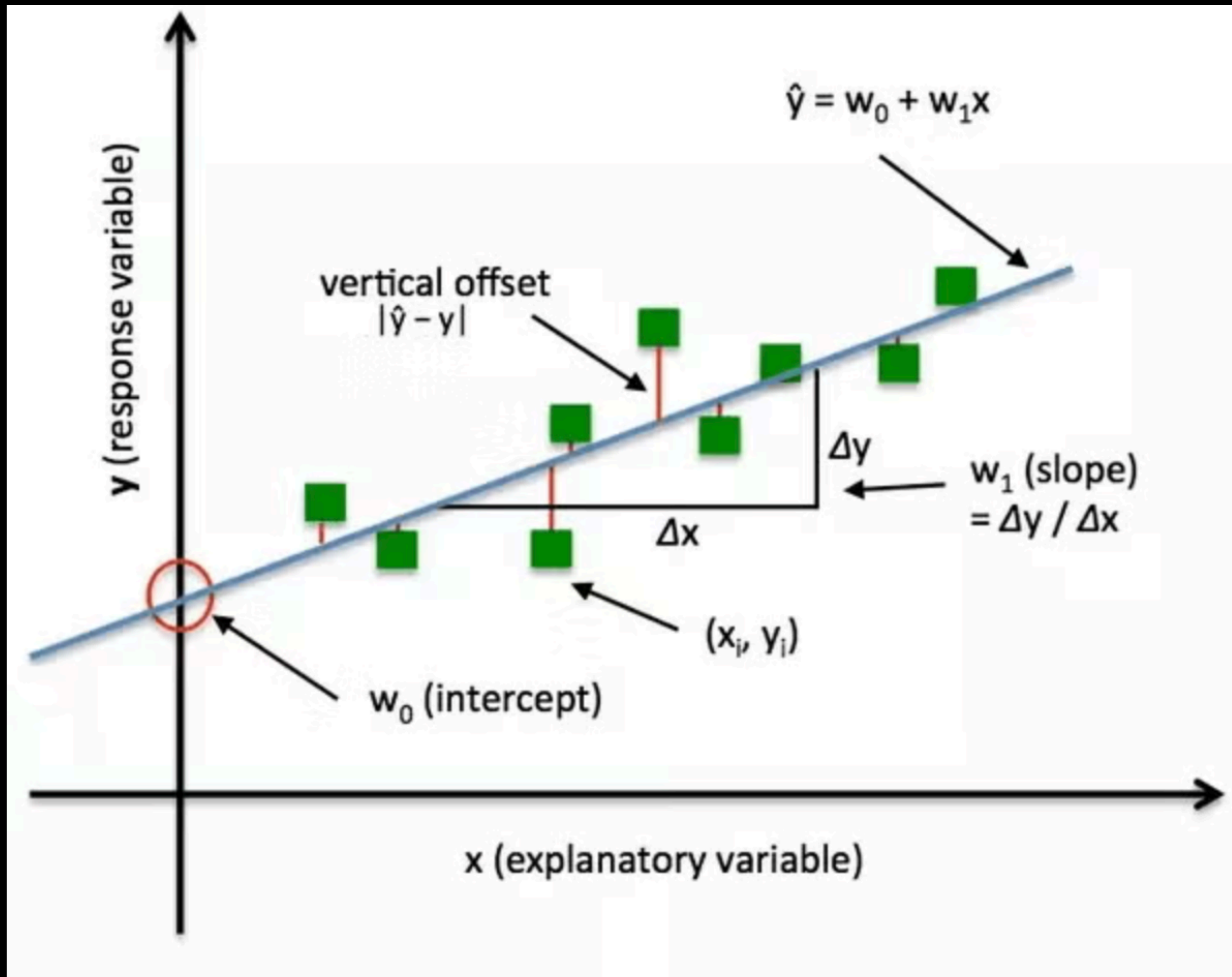
모델을 어떻게 평가하고(손실 함수), 그 평가를 바탕으로 어떻게 파라미터를 개선할지(옵티마이저) 결정합니다.

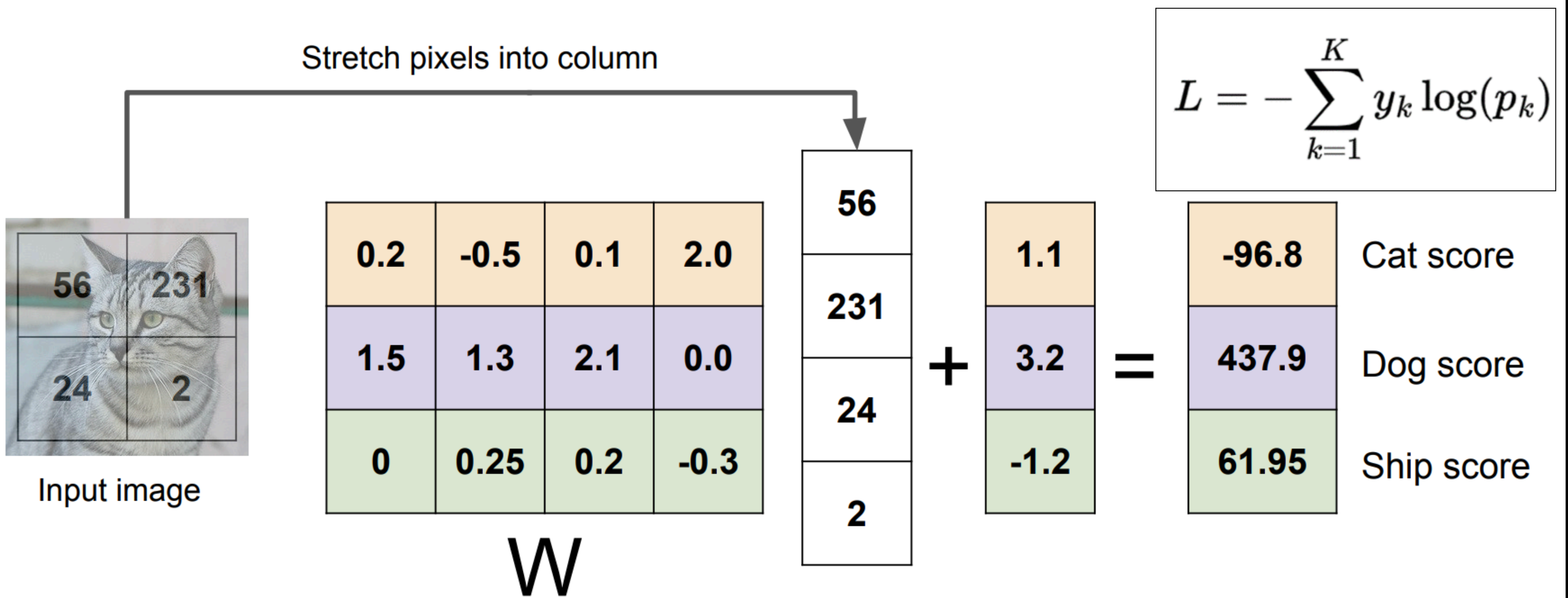
```
criterion = nn.CrossEntropyLoss()  
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Python

코드 해설

- **criterion = nn.CrossEntropyLoss()**: 손실 함수(Loss Function)를 정의합니다. **criterion**은 "기준"이라는 뜻으로, 모델의 예측이 정답과 얼마나 다른지를 측정하는 기준이 됩니다.
 - **CrossEntropyLoss**는 다중 클래스 분류(multi-class classification) 문제에 가장 널리 사용되는 손실 함수입니다.
 - **(중요)** **nn.CrossEntropyLoss**는 내부에 **LogSoftmax**와 **NLLLoss**를 포함하고 있습니다. 따라서 모델의 마지막 레이어에서 직접 **softmax** 함수를 적용할 필요가 없으며, 오히려 적용하면 학습이 제대로 되지 않습니다. 모델은 클래스별 점수(logit)까지만 출력하면 됩니다.
- **optimizer = optim.Adam(...)**: 옵티마이저(Optimizer)를 정의합니다. 옵티마이저는 계산된 손실(loss)을 바탕으로 모델의 파라미터(가중치와 편향)를 어떤 방향과 크기로 업데이트할지 결정합니다.
 - **model.parameters()**: 옵티마이저에게 업데이트할 파라미터가 무엇인지 알려줍니다. **nn.Module**을 상속받았기 때문에 이 메소드 하나로 모델의 모든 학습 가능한 파라미터를 가져올 수 있습니다.
 - **lr=0.001**: 학습률(Learning Rate)입니다. 파라미터를 업데이트할 보폭(step size)을 의미합니다. 너무 크면 학습이 불안정하고, 너무 작으면 학습이 매우 느려집니다. 적절한 값을 찾는 것이 중요하며, 0.001은 Adam 옵티마이저에서 흔히 사용되는 시작 값입니다.





4-2. 훈련 루프 (Training Loop)

PyTorch 학습의 심장부입니다. 데이터셋을 여러 번 반복해서 보면서 모델을 점진적으로 개선합니다.

```
epochs = 3

for epoch in range(epochs):
    model.train() # 훈련 모드 설정
    pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{epochs}")

    for data, target in pbar:
        data, target = data.to(device), target.to(device)

        # 1. 순전파 (Forward Pass)
        output = model(data)

        # 2. 손실 계산
        loss = criterion(output, target)

        # 3. 역전파 (Backward Pass)
        optimizer.zero_grad() # 이전 배치의 그래디언트 초기화
        loss.backward()       # 손실에 대한 각 파라미터의 그래디언트 계산

        # 4. 파라미터 업데이트
        optimizer.step()      # 계산된 그래디언트를 이용해 파라미터 업데이트

    pbar.set_postfix({'loss': f'{loss.item():.4f}'})
```

Python

코드 해설

- **epochs = 3**: 전체 데이터셋을 총 몇 번 반복해서 학습할지를 의미합니다. 1 에폭은 전체 학습 데이터를 한 번 모두 사용하는 것을 뜻합니다.
- **model.train()**: 모델을 '훈련 모드'로 설정합니다. 이는 드롭아웃(Dropout)이나 배치 정규화(Batch Normalization) 같이 훈련 시와 평가 시에 다르게 동작해야 하는 레이어들을 위해 필수적입니다. (이 예제에는 없지만 좋은 습관입니다.)
- **for data, target in pbar::** DataLoader 에서 batch_size 만큼의 이미지 (data)와 정답 레이블 (target)을 가져와 반복합니다.
- **data, target = data.to(device), target.to(device)**: 데이터와 레이블을 GPU/MPS로 보냅니다.

5. 평가 루프 (Evaluation Loop)

한 에폭의 학습이 끝나면, 학습에 사용되지 않은 테스트 데이터로 모델의 현재 성능을 평가합니다.

```
model.eval() # 평가 모드 설정
correct = 0
total = 0

with torch.no_grad(): # 그래디언트 계산 비활성화
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        _, predicted = torch.max(output.data, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()

print(f"Test Accuracy: {100 * correct / total:.2f}%")
```

Python

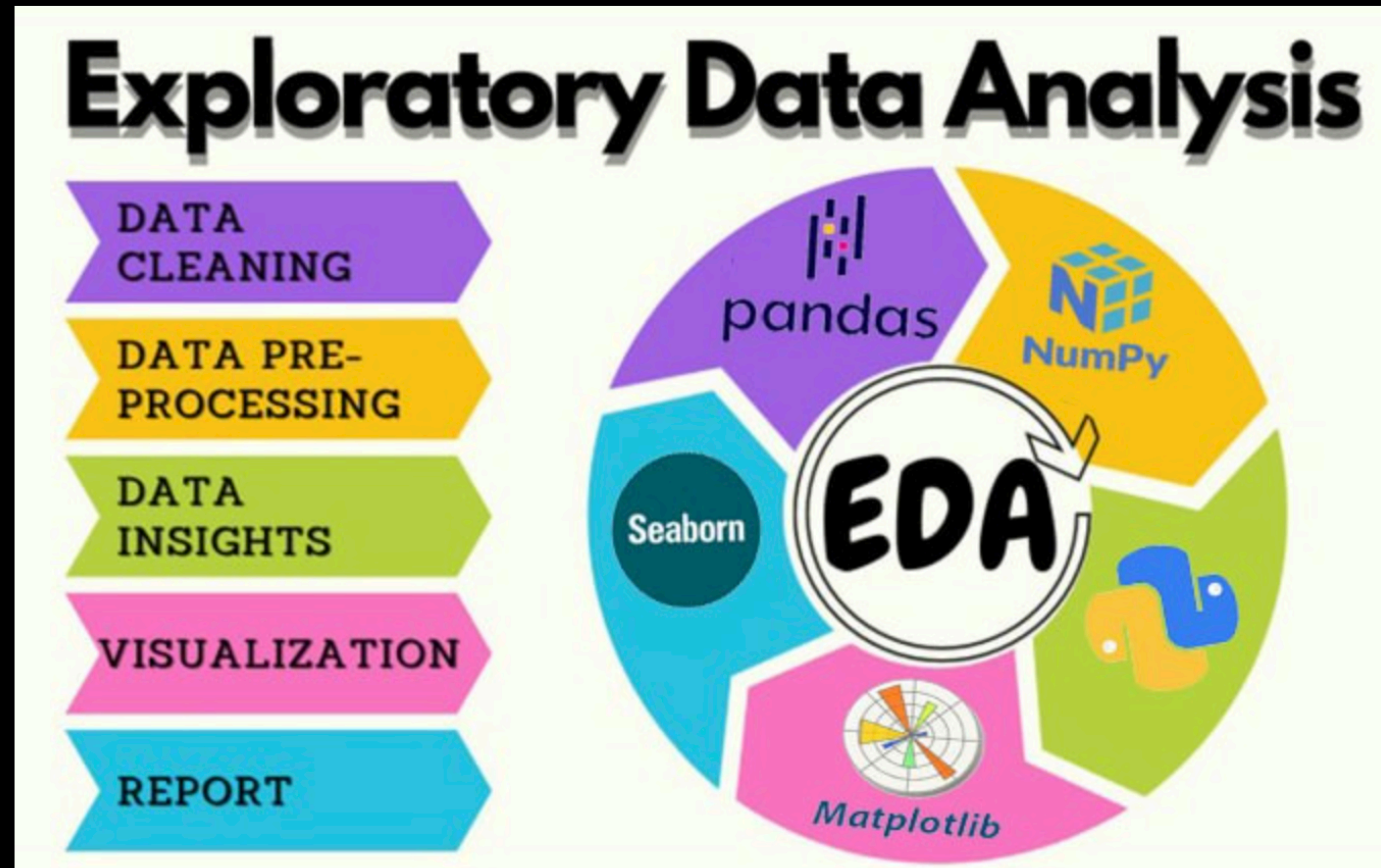
코드 해설

- `model.eval()`: 모델을 '평가 모드'로 설정합니다. `model.train()` 과 반대로, 드롭아웃이나 배치 정규화 레이어가 평가 시에 일관된 방식으로 동작하도록 합니다.
- `with torch.no_grad():`: 이 블록 안에서는 기울기 계산을 하지 않도록 설정합니다. 평가는 모델을 업데이트하는 과정이 아니므로 기울기를 계산할 필요가 없습니다. 이를 통해 불필요한 연산을 줄이고 메모리 사용량을 아낄 수 있습니다.
- `_, predicted = torch.max(output.data, 1)`: 모델이 출력한 `output` (logit)은 (배치 크기, 10) 모양의 텐서입니다. 각 행(이미지)에서 가장 큰 값을 가진 열의 인덱스가 모델의 예측 클래스가 됩니다. `torch.max` 는 최댓값과 그 인덱스를 함께 반환하는데, 우리는 인덱스만 필요하므로 `_` 로 최댓값은 무시하고 인덱스(`predicted`)만 가져옵니다. `dim=1` 은 행(row)을 따라 최댓값을 찾으라는 의미입니다.
- `(predicted == target).sum().item()`: 예측과 정답이 일치하는 개수를 세어 누적합니다.

3. EDA, 전처리, Feature Engineering

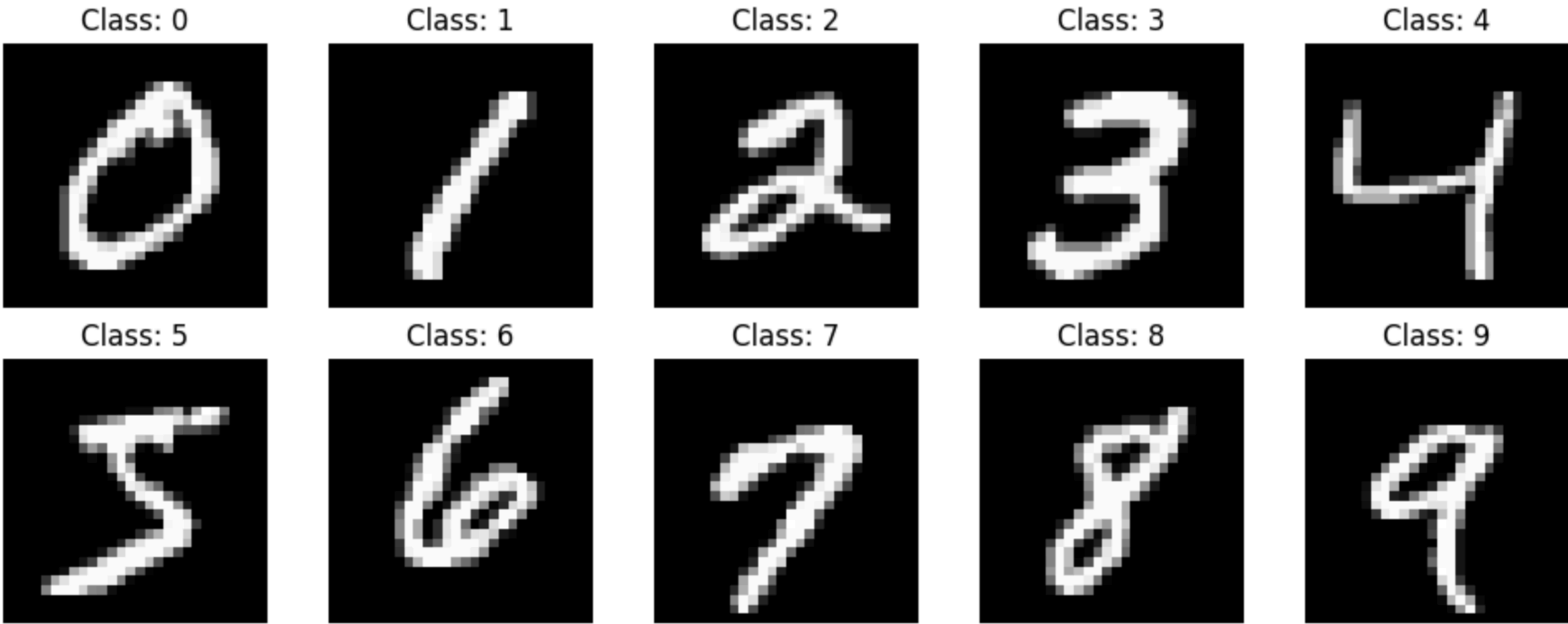
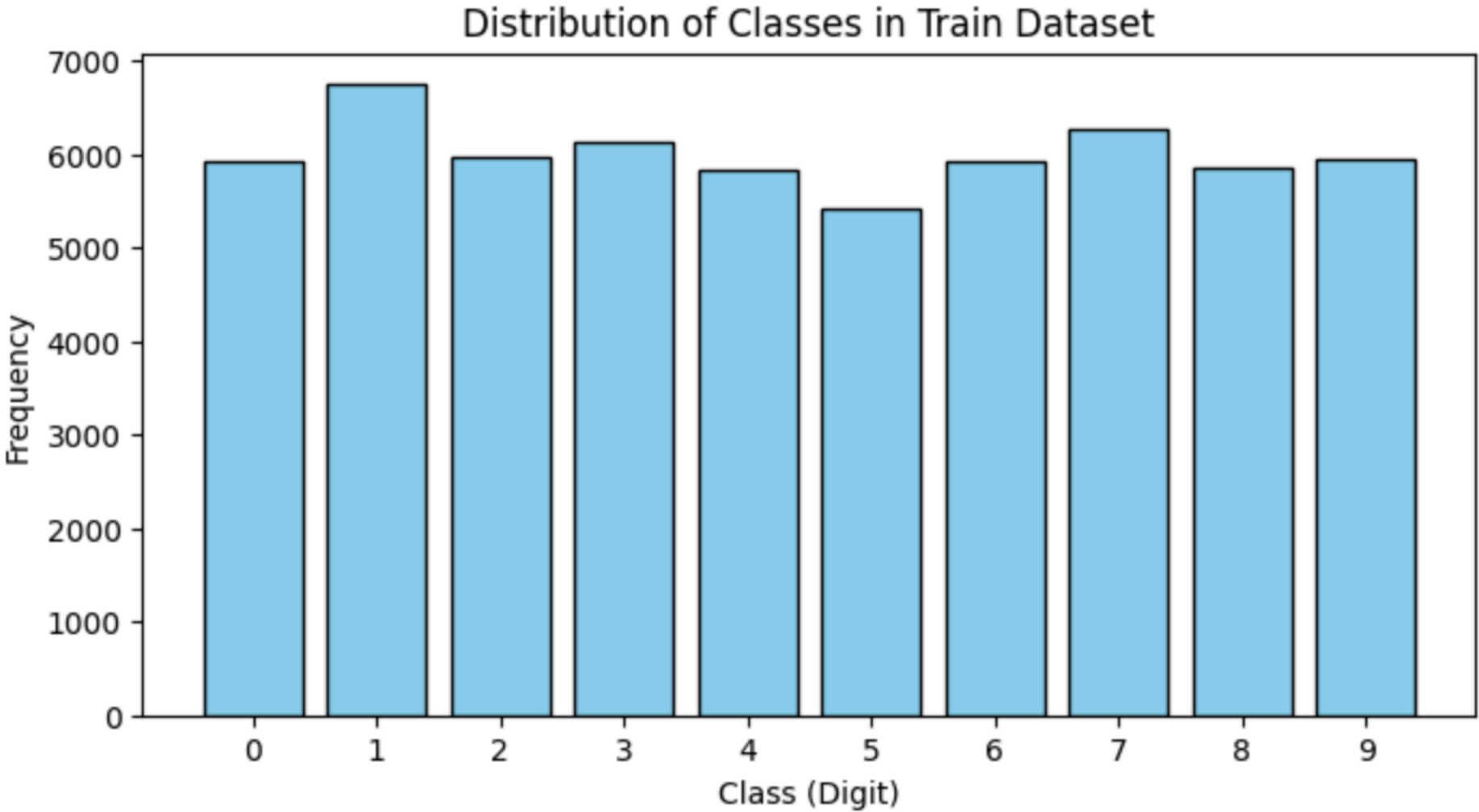
- 설정하는대로 변화하는 모델의 특징을 알아봅니다

EDA

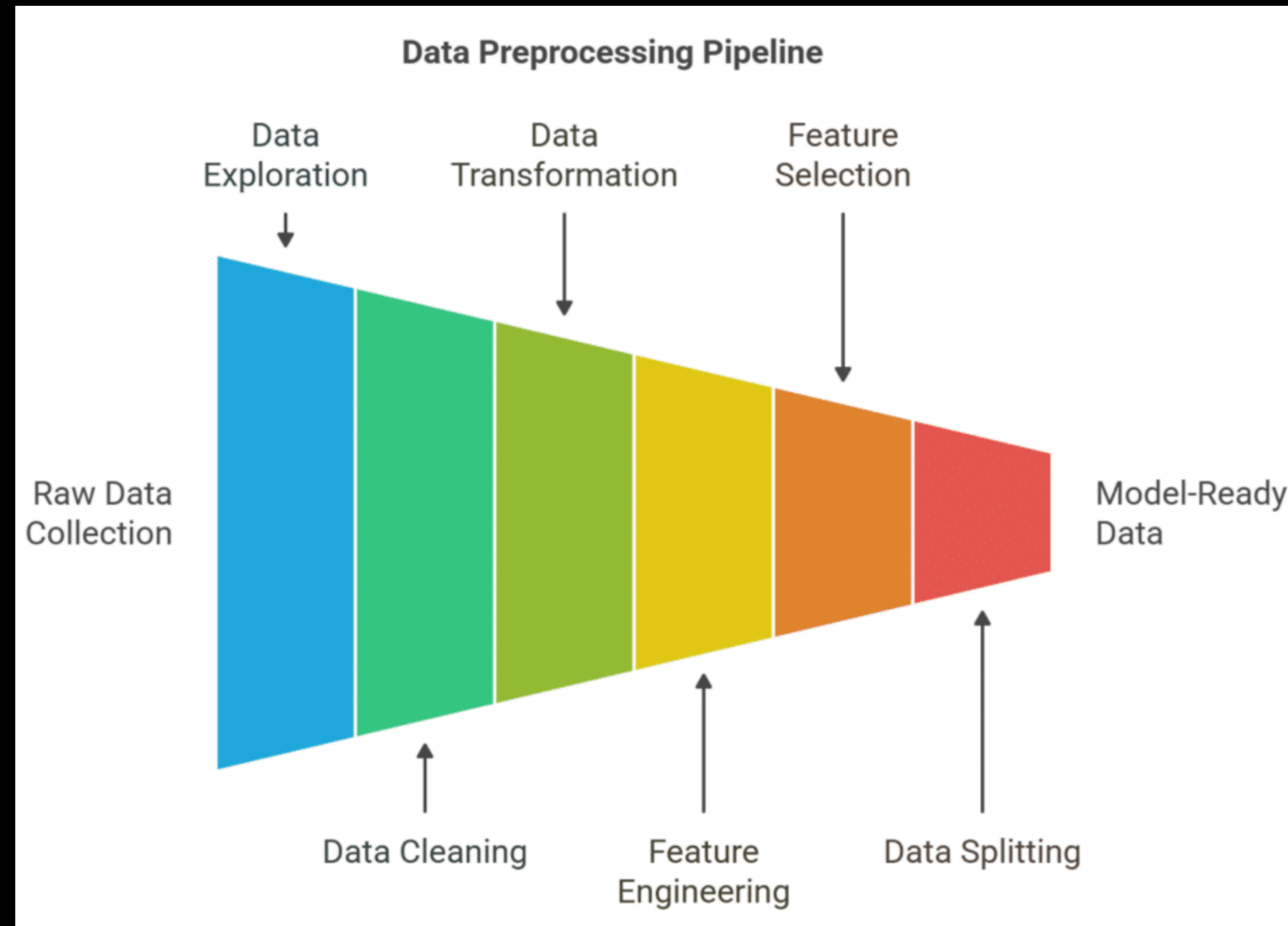


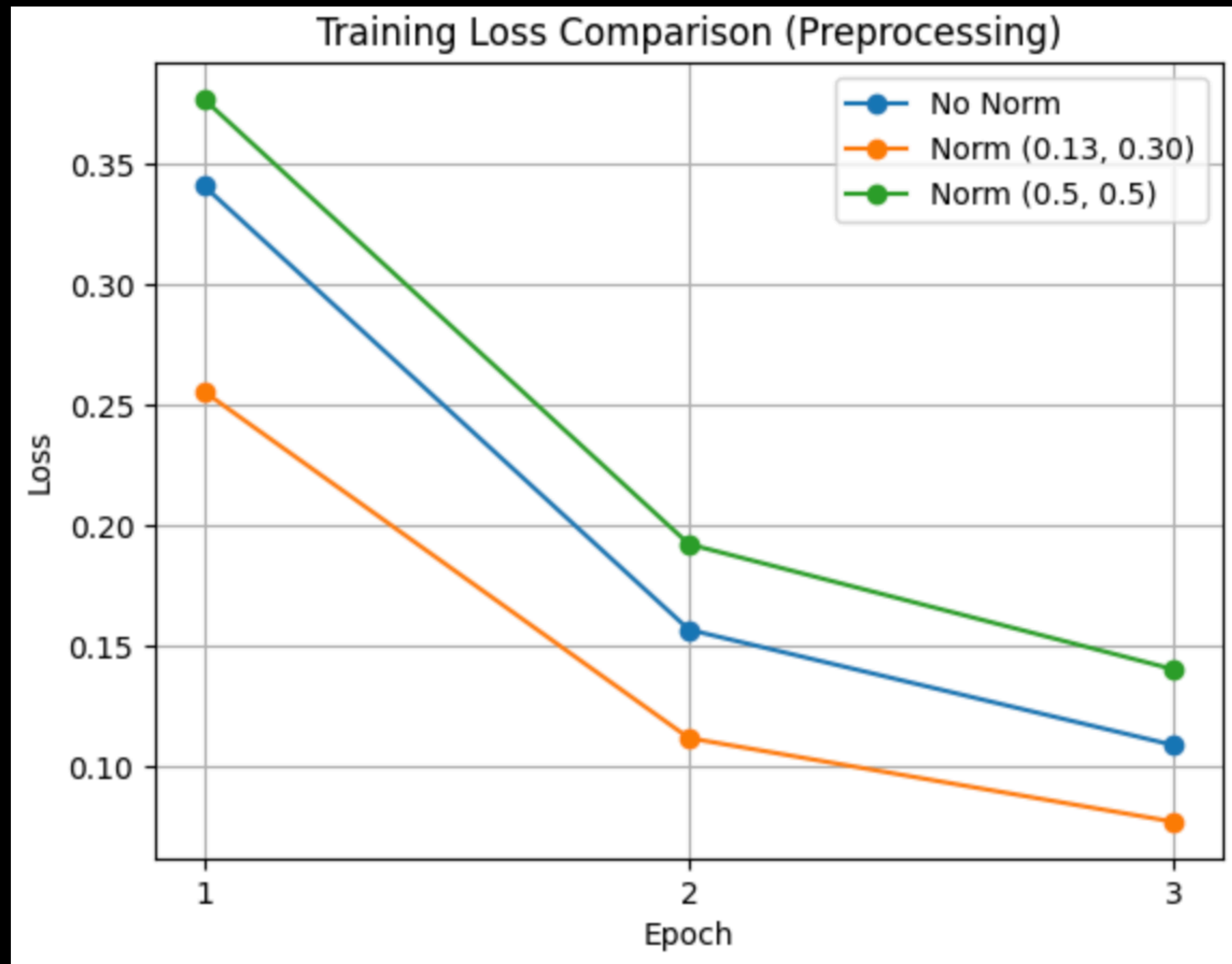
Train dataset size: 60000
Test dataset size: 10000
Image mode: L, Image size: (28, 28)

Train Data Mean: 0.1307
Train Data Std: 0.3081

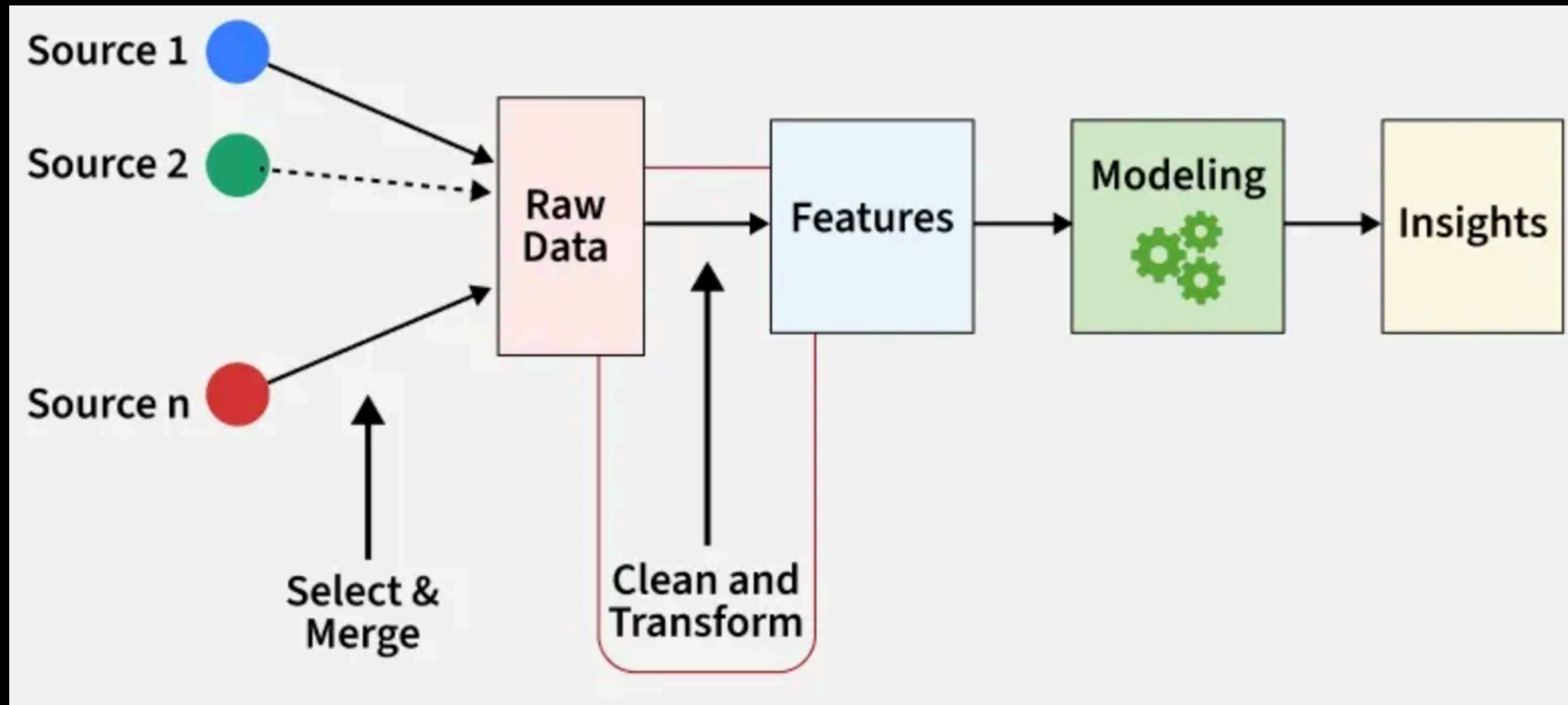


Preprocessing





Feature Engineering



```
print("\n--- 5-2. 피쳐 엔지니어링 (이진화) ---")
transform_binarize = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda x: (x > 0.5).float()), # 0.5 기준으로 0 또는 1
    transforms.Normalize((0.1307,), (0.3081,))
])
_, acc_binarize = train_and_evaluate(transform_binarize, transform_with_prep)
print(f"피쳐 엔지니어링 (이진화) - Test Accuracy: {acc_binarize:.2f}%")
```

--- 5-2. 피쳐 엔지니어링 (이진화) ---

Epoch 1/3: 100%		938/938 [00:18<00:00, 53.24it/s, loss=0.1646]
Epoch 2/3: 100%		938/938 [00:19<00:00, 52.89it/s, loss=0.0163]
Epoch 3/3: 100%		938/938 [00:18<00:00, 51.82it/s, loss=0.0808]

피쳐 엔지니어링 (이진화) - Test Accuracy: 97.48%

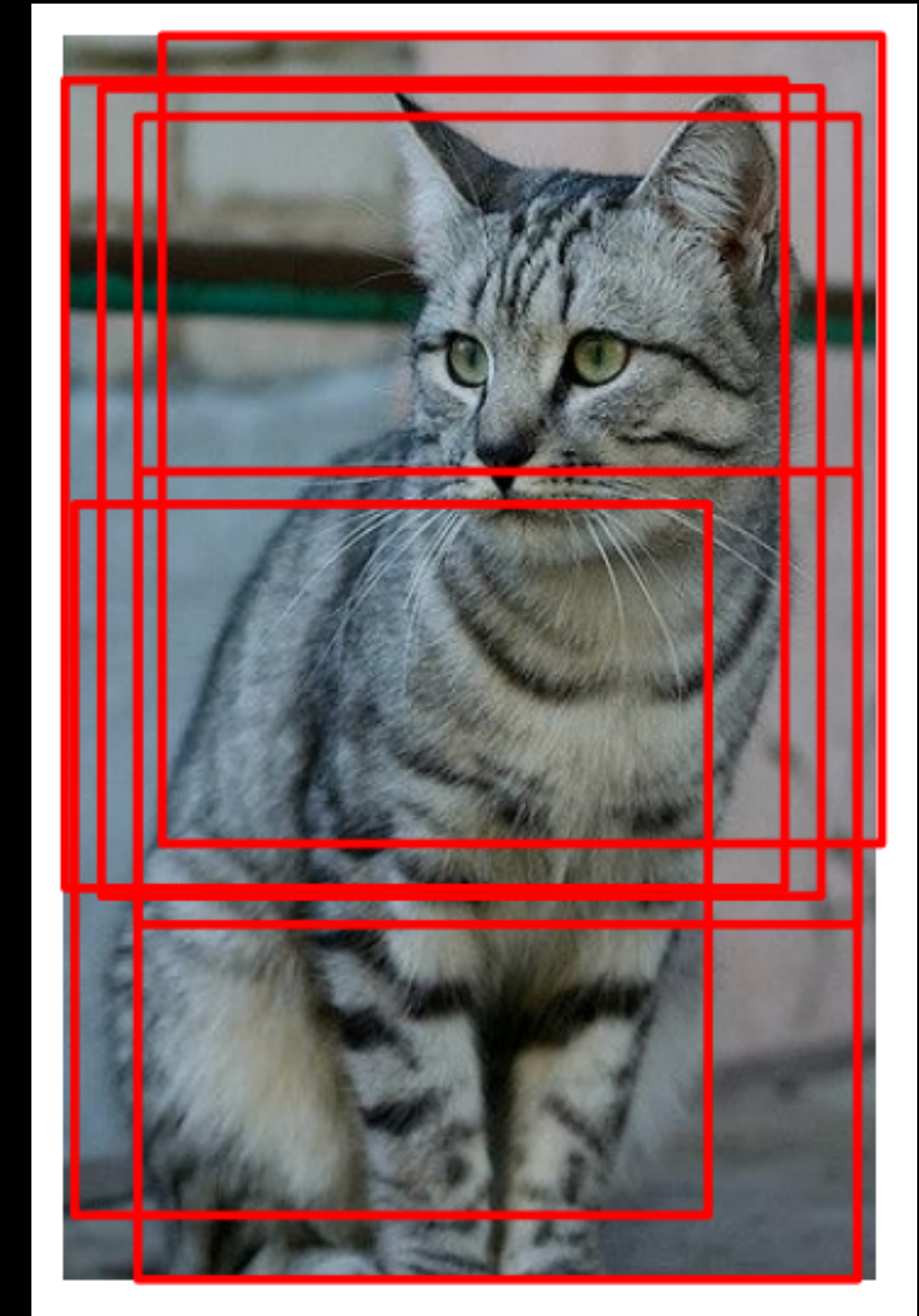
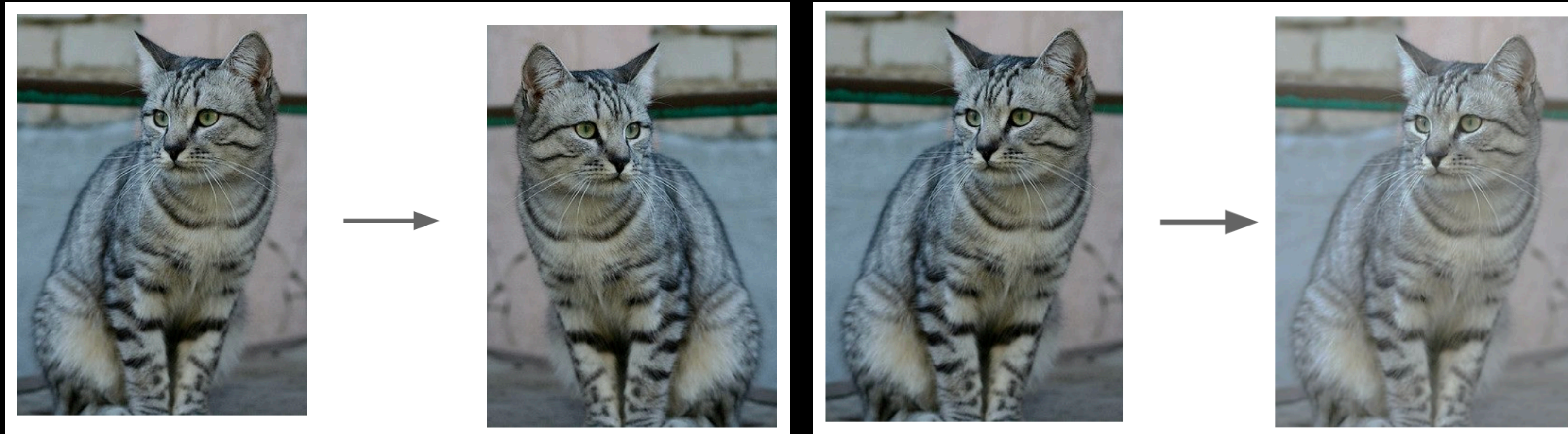
```
print("\n--- 5-3. 피쳐 엔지니어링 (색상 반전) ---")
transform_invert = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda x: 1.0 - x), # 1에서 빼서 색상 반전
    transforms.Normalize((0.1307,), (0.3081,))
])
_, acc_invert = train_and_evaluate(transform_invert, transform_with_prep)
print(f"피쳐 엔지니어링 (색상 반전) - Test Accuracy: {acc_invert:.2f}%")
```

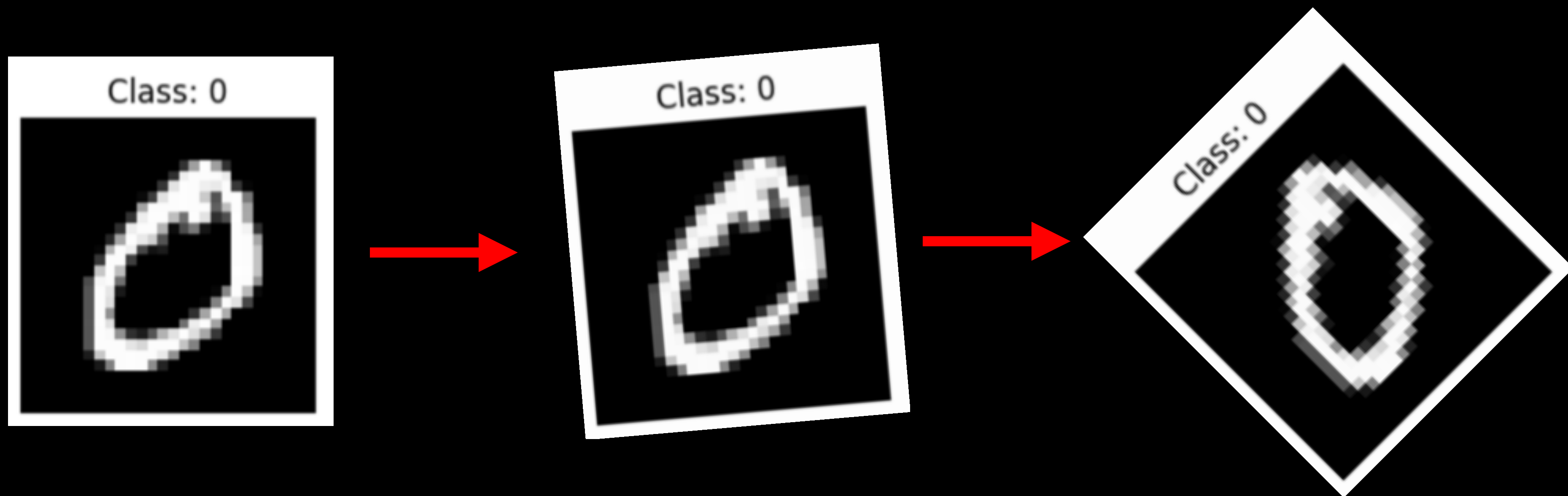
--- 5-3. 피쳐 엔지니어링 (색상 반전) ---

Epoch 1/3: 100%		938/938 [00:19<00:00, 52.41it/s, loss=0.2806]
Epoch 2/3: 100%		938/938 [00:18<00:00, 49.94it/s, loss=0.4143]
Epoch 3/3: 100%		938/938 [00:19<00:00, 52.78it/s, loss=0.1025]

피쳐 엔지니어링 (색상 반전) - Test Accuracy: 0.40%

Data Augmentation





Q & A

End of Document